Digital Signal Processing – Assignment 2
FIR Filters

William Chambers (GUID: 2906463C)
Saad Aziz (GUID: 2917444A)

University of Glasgow, School of Engineering

November 8, 2026

1 Recordings

[State who recorded the ECGs in the lab, the electrode placement/lead setup, the sampling rate f_s (calculated from column 0 timestamps) and corresponding Nyquist frequency $f_s/2$, which column of the TSV contains the ECG channel (column 8, 0-indexed), and why each partner's recording was selected (e.g. primary recording for Tasks 1–4 vs. partner's recording for the Task 5 matched-filter template).]

2 Task 1: Time and Frequency Analysis

[Briefly state how the time axis and single-sided linear frequency spectrum ($X = \frac{1}{N}\,\mathrm{FFT}(x)$ up to $f_s/2$) were computed from the raw recording without using prohibited functions like `fftfreq`. Emphasise that the frequency domain is plotted on a **linear axis** as required by the brief.]

[Insert vector Figure (PDF/EPS/SVG): Raw ECG in the time domain showing (1) the full recording to reveal overall baseline drift/offset, and (2) a zoomed-in trace of a single heartbeat annotated (e.g. via Inkscape) with the P, Q, R, S, and T waves and proper axis labels/units.]

Figure 1: [Caption specifying the recording, full time-domain trace, and zoomed single heartbeat with labelled PQRST waves.]

[Insert vector Figure (PDF/EPS/SVG): Raw ECG in the frequency domain on **linear axes** (both full magnitude view and zoomed-in magnitude view if needed to resolve smaller peaks), annotated (e.g. via Inkscape) to point out DC/baseline wander, heart-rate fundamental and harmonics, 50 Hz mains interference, and higher-frequency noise floor.]

Figure 2: [Caption describing the annotated linear-axis frequency spectrum of the raw ECG.]

ECG signal and harmonics. [Measure the beat-to-beat (R–R) period from the time-domain plot, calculate the corresponding fundamental heart rate frequency $f_0 = 1/T_{\mathrm{RR}}$, locate the fundamental peak and its integer harmonics ($2f_0, 3f_0, \ldots$) in the zoomed frequency spectrum, and identify the frequency band where the diagnostic PQRST energy resides.]

Noise and interferences. [Identify and explain every non-ECG component in both domains: (1) DC offset and low-frequency baseline wander (respiration/movement drift in time domain, large spike near 0 Hz in spectrum), (2) 50 Hz mains power-line hum (and state whether any

100 Hz harmonic is present below the 125 Hz Nyquist limit), and (3) broadband muscle (EMG) / electronic noise floor and where it overlaps or sits above the ECG spectrum.]

Frequencies to remove. [Explicitly list the exact frequency ranges chosen for removal (in Hz) and justify **why** each cut-off was chosen using your harmonic analysis: why the low cut-off removes DC and baseline wander without attenuating the fundamental heart rate f_0, and why the upper bandstop/lowpass cut-off removes 50 Hz mains hum (and high-frequency noise) without rounding or distorting the sharp QRS complex.]

3 Task 2: FIR Filter Coefficient Design

Number of coefficients. [Give the exact mathematical formula and reasoning for how the number of taps M is automatically calculated inside your general-purpose design function from f_s and the array of frequency ranges to remove (e.g. finding the narrowest stopband width $\Delta f_{\min}$, setting the required frequency resolution / bin width δf, computing $M = f_s/\delta f$, and enforcing even parity if required for symmetric half-swapping). Discuss the engineering trade-off between frequency resolution (sharper cut-offs with larger M) versus filter delay ($M/2$ samples) and realtime computational cost.]

IFFT coefficient calculation. [Explain step-by-step how the general-purpose function computes the FIR coefficients: (1) constructing the ideal frequency response array $H[k]$ of length M with passbands set to 1 and target stopband ranges set to 0, (2) maintaining conjugate/Hermitian symmetry across positive and negative frequencies and explicitly handling edge cases such as DC (0 Hz, bin 0) and Nyquist ($f_s/2$), (3) applying the IFFT and taking the real part to obtain the impulse response $h[n]$, (4) shifting/swapping the two halves of $h[n]$ so the impulse response is causal and centred at $M/2$ rather than split across the ends, and (5) multiplying by a window function (e.g. Hamming) to suppress Gibbs phenomenon ringing caused by truncating the impulse response.]

[Insert vector Figure (PDF/EPS/SVG): Plot of the designed FIR filter showing (1) the final shifted and windowed impulse response / tap coefficients $h[n]$ against tap index n, and (2) the resulting frequency response magnitude $|H(f)|$ verifying the stopbands at DC/baseline wander and 50 Hz.]

Figure 3: [Caption stating the calculated number of taps M, the removed frequency ranges, the window used, and what the time/frequency plots show.]

4 Task 3: Realtime FIR Filtering

Implementation. [Explain the architecture of `class FIRfilter` with constructor `__init__(self, _coefficients)` and method `dofilter(self, v)`. Emphasise that `dofilter` accepts a single scalar input sample $x[n]$ and returns a single scalar output sample $y[n] = \sum_{m=0}^{M-1} h[m]\, x[n-m]$ to simulate sample-by-sample realtime processing. Explicitly explain how the delay line is implemented efficiently (e.g. using a circular ring buffer and a moving index pointer/offset rather than shifting all M array elements at every time step, which the brief warns loses marks).]

Filtered ECG results. [Show the result of feeding the raw ECG sample-by-sample through `dofilter`. Explain the initial transient / buffer-fill period (M samples) and the linear-phase group delay ($M/2$ samples). Verify and explicitly comment on: (1) complete removal of DC offset and baseline wander in the full time trace, (2) removal of the 50 Hz hum in the time and

frequency domains, and (3) visual confirmation on a zoomed single heartbeat that the P, Q, R, S, and T waves are intact, sharp, and undistorted.]

[Insert vector Figure (PDF/EPS/SVG): Task 3 realtime FIR filtered ECG showing (1) the full filtered time-domain trace (flat baseline around 0 V), (2) a zoomed-in single heartbeat annotated with P, Q, R, S, T to prove the morphology is intact and undistorted, and (3) the linear frequency spectrum of the filtered ECG confirming the stopbands.]

Figure 4: [Caption describing the sample-by-sample FIR filtered ECG in the time domain (full and zoomed single beat with PQRST) and frequency domain.]

5 Task 4: Adaptive LMS Filter

Implementation. [Explain how `doFilterAdaptive(self, signal, noise, learningRate)` is added to `FIRfilter` and operates sample-by-sample on scalars: (1) what the primary input `signal` $d[n]$ is (and if DC/baseline wander is removed beforehand), (2) how the scalar 50 Hz reference `noise` $x[n] = \sin(2\pi \cdot 50\, n/f_s)$ is generated and filtered by the adaptive weights to estimate the interference $y[n]$, (3) how the cleaned ECG error output $e[n] = d[n] - y[n]$ is computed, and (4) how each filter weight w_k is updated at every step ($w_k \leftarrow w_k + \mu\, e[n]\, x[n-k]$). Justify your choice of filter length M_{LMS} and learning rate μ (trade-off between adaptation speed and stability/excess error).]

Comparison with FIR filter design. [Compare the adaptive LMS filter output directly against the Task 3 fixed FIR bandstop filter: compare (1) 50 Hz noise suppression quality and PQRST waveform preservation on a zoomed heartbeat, (2) startup behaviour (LMS weight convergence time vs. fixed FIR buffer fill and group delay), (3) computational efficiency (number of taps required for LMS vs. the large M needed for the fixed FIR filter), and (4) frequency selectivity (narrow notch tracking the reference vs. wider fixed bandstop).]

[Insert vector Figure (PDF/EPS/SVG): Direct visual comparison of the Task 4 adaptive LMS output and the Task 3 fixed FIR output, including (1) full or startup time trace showing LMS convergence, (2) zoomed single heartbeat overlay showing PQRST fidelity of both methods, and (3) frequency spectrum comparison if applicable.]

Figure 5: [Caption comparing the adaptive LMS filter result (μ, M_{LMS}) with the Task 3 fixed FIR filter design.]

6 Task 5: ECG Heartbeat Detection

Matched filter template. [Explain how a single clean PQRST heartbeat was isolated from the **other lab partner's** filtered ECG recording to act as a real ECG template. Explain the theory and implementation of the matched filter: why the template array is time-reversed ($h[n] = \text{template}[N-1-n]$) and passed into a `FIRfilter` instance so that sample-by-sample convolution via `dofilter` performs cross-correlation in realtime without using prohibited functions like `correl` or `conv`. Mention any post-processing applied to the matched filter output (e.g. squaring) to enhance the R-peak SNR.]

[Insert vector Figure (PDF/EPS/SVG): The single heartbeat template extracted from the other lab partner's ECG (and/or its time-reversed matched filter impulse response) with proper time and amplitude labels.]

Figure 6: [Caption identifying which partner's recording the template came from, the time window extracted, and the resulting matched filter coefficients.]

R-peak detection and momentary heart rate. [Explain how R-peaks are detected from the matched filter output (e.g. threshold crossing / local peak detection) and how the momentary heart rate is calculated from the inverse intervals between consecutive R-peaks ($\mathrm{HR} = 60/\Delta t_{\mathrm{RR}}$ in bpm). Explicitly detail the heuristic(s) implemented to reject false detections and artefacts (e.g. minimum refractory period corresponding to a maximum physiological heart rate of $\leq 200\,\mathrm{bpm}$, minimum heart-rate bound, and/or ignoring the initial filter startup transient) and discuss the resulting heart rate trace over time.]

[Insert vector Figure (PDF/EPS/SVG): (1) Matched filter output (and/or ECG trace) showing the detected R-peaks, and (2) the momentary heart rate (in bpm or bps/Hz, clearly labelled) plotted against time after heuristic filtering.]

Figure 7: [Caption describing the R-peak detections and the momentary heart rate vs. time plot with heuristic outlier removal.]

A Acknowledgement of GenAI Use

Tools: [List every GenAI tool used with its exact name, version, and publisher (e.g. Claude Opus 5.5 (Anthropic); Gemini (Google); GitHub Copilot (Microsoft)) as required by the brief and UoG guidance.]
Use: [State clearly (1) how each GenAI tool was used (e.g. LaTeX report framework, debugging, syntax lookups, code optimisation), and (2) explicitly what work you contributed yourself (e.g. lab ECG recordings, spectral analysis, filter design logic, ring-buffer and LMS implementations, matched filter and heuristic design, and writing the report).]

B Usage instructions

[Provide exact platform-independent command-line instructions to run the code from a terminal (not Spyder) after unzipping the submission, list required Python packages (`numpy`, `scipy`, `matplotlib`), and confirm that running the script(s) displays all plots without crashing and uses only relative paths.]

C Python Code

[Before submitting, verify that: (1) every Python file in the submission is included below, (2) no prohibited high-level functions (`fftfreq`, `lfilter`, `firwin`, `conv`, `correl`, FFT/IIR filtering, or array-in/array-out a-causal filtering) appear anywhere, (3) no absolute file paths exist, and (4) any LLM-generated test/debug bloat is removed or optimised so every line is concise, human-readable, and explainable in an interview.]

C.1 support_funcs.py

```python
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR = Path(__file__).resolve().parent


def import_ecg_data(filename):
    """
    Parameters:
    filename: str or Path
        The name of the file containing the ECG data.

    Returns:
    tuple[np.ndarray, np.ndarray, float]
        (t, ecg, sample_rate) - time vector (s), ECG signal (V), and sampling rate (Hz).
    """
    ECG_COLUMN = 8  # ADC channel 2
    data = np.loadtxt(BASE_DIR / filename)
    t = data[:, 0]
    ecg = data[:, ECG_COLUMN]
    sample_rate = 1 / (t[1] - t[0])  # fs = 250 Hz

    return t, ecg, sample_rate


def vector_plot_export(savename):
    """
    Exports the current matplotlib figure as vector graphics (.svg and .pdf) to Report/figs/.

```

```
    Parameters:
    savename: str or Path
        Output file name (without extension).
    """
    figs_dir = BASE_DIR / "Report" / "figs"
    figs_dir.mkdir(parents=True, exist_ok=True)
    stem = Path(savename).stem
    plt.savefig(figs_dir / f"{stem}.svg", format="svg", bbox_inches="tight")
    plt.savefig(figs_dir / f"{stem}.pdf", format="pdf", bbox_inches="tight")
```

C.2 ecg_plot.py

```
'''
Plot the ECG in the time- and frequency domain (linear axis) and identify
the ECG and the noise in the spectrum. There is certainly baseline
wander and 50Hz interference but there might be other interferences.
Decide which frequency components need to be removed. Identify these
frequencies representing noise using your judgement and knowledge
about harmonics
'''

import numpy as np
from scipy.fft import fft
import matplotlib.pyplot as plt

from support_funcs import import_ecg_data, vector_plot_export

# ------ TASK 1 PLOTS ------
# Function to plot ECG signal in the time and frequency domain
def plot_ecg(name, sample_rate, ecg, beat_start, beat_end):
    n = len(ecg)
    time = np.arange(n) / sample_rate # array of times

    # Frequency domain (linear)
    ecg_fft = (1 / n) * fft(ecg)
    mag_fft = np.abs(ecg_fft)
    freqs = np.arange(n // 2) * sample_rate / n # frequency bins up to sample_rate/2

    plt.figure(figsize=(12, 8))
    plt.suptitle(f"ECG Analysis: {name}")

    # whole recording
    plt.subplot(2, 1, 1)
    plt.title("time domain")
    plt.plot(time, ecg)
    plt.xlim(0, n / sample_rate)
    plt.xlabel("time (s)")
    plt.ylabel("amplitude (V)")

    # one heartbeat (PQRST)
    plt.subplot(2, 1, 2)
    plt.title("one heartbeat")
    plt.plot(time, ecg)
    plt.xlim(beat_start, beat_end)
    plt.xlabel("time (s)")
    plt.ylabel("amplitude (V)")

    plt.tight_layout() # stop axes overlapping
    vector_plot_export("t1_time")

    # --- figure 2: frequency domain ---
    plt.figure(figsize=(12, 8))
    plt.suptitle(f"ECG spectrum: {name}")
```

```
52
53      plt.subplot(2, 1, 1)
54      plt.title("full spectrum")
55      plt.plot(freqs[0:], mag_fft[0:n // 2])
56      plt.xlim(0, sample_rate / 2)
57      plt.xlabel("frequency (Hz)")
58      plt.ylabel("magnitude (V)")
59
60      # zoomed in spectrum
61      plt.subplot(2, 1, 2)
62      plt.title("zoomed-in spectrum")
63      plt.plot(freqs[0:], mag_fft[0:n // 2])
64      plt.xlim(0, sample_rate / 4)
65      plt.ylim(0, 3e-5) # reveals the small peaks (TUNE IT)
66      plt.xlabel("frequency (Hz)")
67      plt.ylabel("magnitude (V)")
68
69      plt.tight_layout() # stop axes overlapping
70      vector_plot_export("t1_freq")
71
72  # --- IMPORT ECG DATA ---
73  t, ecg, sample_rate = import_ecg_data("saad_ecg_2.tsv")
74  plot_ecg("Saad 2", sample_rate, ecg, 5.5, 6.7) # pick a window containing a single beat
75  plt.show()
76
77  #TODO: fix plot text, title, legend etc.
```

C.3 fir_design.py

```
1   '''
2   Create a general purpose function which calculates the FIR coefficients
3   by providing the sampling rate and an array of frequency ranges which
4   need to be removed also taking care of edge cases such as DC removal.
5   The design function should use the IFFT to calculate the coefficients and
6   also calculates the number of coefficients. Explain your reasoning of how
7   the number of coefficients are calculated
8   '''
9
10  import numpy as np
11  from scipy.fft import ifft
12
13  #Describe the function
14  def fir_design(sample_rate, ranges):
15
16      ranges = np.array(ranges)
17      min_width = np.min(ranges[:, 1] - ranges[:, 0])
18
19      bin_size = min_width / 2
20      M = int(sample_rate / bin_size) #number of coefficients (explain!)
21      #why does M have to be even?
22      M += M%2 # ensure M is even
23
24      '''
25      The number of coefficients, M, is calculated based on the minimum width of the frequency
        range arguments.
26      The minimum width is divided by 2 so that each range alway has at least 2 bins to operate on.

27      The number of coefficients is then calculated by dividing the sample rate by the bin size.
28      M is made even so that the impulse response can be split into two equal halves when it is
        shifted.
29      '''
30
31      #Using folded axis to create a symmetric frequency response
```

```
    f = np.arange(M) * sample_rate / M #freq. of eahc bin
    f = np.minimum(f, sample_rate - f) #fold: bin M-k gets the same freq. as bin k
    H = np.ones(M) #initialise frequencies to 1 (pass band)

    #define low and high
    for low, high in ranges:
        H[(f >= low) & (f <= high)] = 0 #remove bins in range (set to 0)

    # IFFT - produces the impulse response, keeping only the real part (H is mirrored, so
    complex cancels)
    h = np.real(ifft(H))

    # Swap the two halves (see 6.7.6 or Bernd's notes)
    h_shift = np.zeros(M)
    h_shift[0:M//2] = h[M//2:M]
    h_shift[M//2:M] = h[0:M//2]

    h_windowed = h_shift * np.hamming(M) #can this be used?

    '''
    Edge cases (DC and Nyquist) and covered by the folded axis (bin 0 and bin 125 [fs/2])
    '''
    return h_windowed


###REMOVE THIS POOPY
if __name__ == "__main__":
    import matplotlib.pyplot as plt
    from scipy.fft import fft
    from support_funcs import import_ecg_data, vector_plot_export

    _, _, sample_rate = import_ecg_data("saad_ecg_2.tsv")
    baseline_wander = [0, 0.5]
    mains_hum = [45, 125]

    h = fir_design(sample_rate, [baseline_wander, mains_hum])
    M = len(h)

    freqs_h = np.arange(M // 2 + 1) * sample_rate / M
    H_mag = np.abs(fft(h))[:M // 2 + 1]

    plt.figure(figsize=(12, 6))
    plt.suptitle(f"FIR Filter Design (M = {M} taps, fs = {sample_rate:.0f} Hz)")

    plt.subplot(2, 1, 1)
    plt.title("Impulse Response h[n] (Shifted & Hamming Windowed)")
    plt.plot(np.arange(M), h)
    plt.xlim(0, M - 1)
    plt.xlabel("Tap Index (n)")
    plt.ylabel("Amplitude")
    plt.grid(True)

    plt.subplot(2, 1, 2)
    plt.title("Frequency Response |H(f)|")
    plt.plot(freqs_h, H_mag)
    plt.axvspan(0, 0.5, color="red", alpha=0.2, label="DC / Baseline Stopband (0-0.5 Hz)")
    plt.axvspan(45, 125, color="orange", alpha=0.2, label="Mains / HF Stopband (45-125 Hz)")
    plt.xlim(0, sample_rate / 2)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Gain")
    plt.legend(loc="upper right")
    plt.grid(True)

```

```
    plt.tight_layout()
    vector_plot_export("t2_coeffs")
    plt.show()
```

C.4 fir_filter.py

```
'''
Create a Python FIR filter class which implements an FIR filter which has
a method of the form value dofilter(self,value) where both the value
argument and return value are scalars and not vectors (!) so that it can
be used in a realtime system. The constructor of the class takes the
coefficients as its input:
    class FIRfilter:
        def __init__(self,_coefficients):
            # your code here
        def dofilter(self,v):
            # your code here
        return result
Filter your ECG with the above FIR filter class using the coefficients from
'''
import numpy as np


class FIRfilter:
    def __init__(self, _coefficients):
        """
        Parameters:
        fir_coefficients: array
            The M filter coefficients calculated in Task 2.
        """
        self.coefficients = np.array(_coefficients, dtype=float) # load coefficients into object
        self.ntaps = len(self.coefficients) # get buffer array length

        self.buffer = np.zeros(self.ntaps, dtype=float) # generate buffer array

        self.offset = 0 # set offset to zero, is this necessary? is it not zero by default?

    def dofilter(self, v):
        """
        Parameters:
        v: float
            The incoming input sample x[n].

        Returns:
        float
            The filtered output sample y[n].
        """
        self.buffer[self.offset] = v # load current sample into buffer

        k = self.offset + 1
        # output = sum[0:M-1](h(m) \dot x(n-m))
        output = (
            np.dot(self.coefficients[:k], self.buffer[self.offset::-1])  # fwd sum
            + np.dot(self.coefficients[k:], self.buffer[:self.offset:-1]) # backward sum
        ) # dot product multiply coefficients with samples in object

        # iterate offset position, reset to zero
        if self.offset == self.ntaps - 1:
            self.offset = 0
        else:
            self.offset += 1


```

```
        return float(output)

    def doFilterAdaptive(self, signal, noise, learningRate):
        """
        Parameters:
        signal: float
            Current contaminated ECG sample d[n].
        noise: float
            Current reference noise sample x[n] (e.g. sin(2*pi*50/fs * n)).
        learningRate: float
            LMS step size (mu) controlling how fast the weights adapt.

        Returns:
        float
            The cleaned ECG sample e[n] = d[n] - y[n].
        """
        read_idx = self.offset # store current write index before dofilter increments offset

        y = self.dofilter(noise) # filter reference noise x[n] to estimate interference y[n]

        e = signal - y # subtract estimated noise from signal to get error / clean ECG e[n]

        k = read_idx + 1
        step = learningRate * e # calculate LMS weight update factor (mu * e[n])
        # w_m[n+1] = w_m[n] + mu * e[n] * x[n-m]
        self.coefficients[:k] += step * self.buffer[read_idx::-1] # fwd tap weight update
        self.coefficients[k:] += step * self.buffer[:read_idx:-1] # backward tap weight update

        return float(e)
```

C.5 fir_realtime_sim.py

```
# 3b & 4 - Simulating RT FIR and Adaptive LMS filtering importing fir_design and fir_filter.

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

from fir_design import fir_design
from fir_filter import FIRfilter
from support_funcs import import_ecg_data, vector_plot_export

# --- 1. Load ECG Data ---
t, ecg, sample_rate = import_ecg_data("saad_ecg_2.tsv")
n = len(ecg)

# --- 2. Design Fixed FIR Filter (Task 2 / Task 3) ---
# Frequency ranges to remove:
# - baseline_wander [0, 0.5] Hz: removes DC offset and slow respiration/movement drift.
# - mains_hum [45, 125] Hz: removes 50 Hz powerline interference and high-frequency noise
#   up to the Nyquist frequency (fs / 2 = 125 Hz), which is the maximum representable frequency.
baseline_wander = [0, 0.5]
mains_hum = [45, 125]

# fir_design calculates M = fs / (min_width / 2) = 250 / 0.25 = 1000 taps (0.25 Hz bin width),
# sets stopband bins to 0 on a folded symmetric frequency axis, takes the real IFFT,
# shifts the impulse response to be causal (centred at M/2), and applies a Hamming window.
h = fir_design(sample_rate, [baseline_wander, mains_hum])
M = len(h)

# --- 3. Set Up Adaptive LMS Filter (Task 4) ---
# Pre-remove DC / baseline wander (0 - 0.5 Hz) so the 50 Hz LMS canceller operates on a zero-
    mean ECG.
```

```
h_hp = fir_design(sample_rate, [baseline_wander])
hp_filter = FIRfilter(h_hp)

M_lms = 10               # Number of adaptive filter taps (1 period of 50 Hz at 250 Hz is 5 samples)
learning_rate = 0.01     # LMS step size (mu)
f_noise = 50.0           # 50 Hz mains interference reference frequency
lms_filter = FIRfilter(np.zeros(M_lms))

# --- 4. Real-Time Sample-by-Sample Filtering (Simple FIR & Adaptive LMS) ---
fir = FIRfilter(h)
ecg_filtered = np.zeros(n)
ecg_hp = np.zeros(n)
ecg_lms = np.zeros(n)

for i in range(n):
    # Simple fixed FIR filter (Task 3)
    ecg_filtered[i] = fir.dofilter(ecg[i])

    # Adaptive LMS filter (Task 4): high-pass filter first, then cancel 50 Hz interference
    ecg_hp[i] = hp_filter.dofilter(ecg[i])
    noise_ref = np.sin(2 * np.pi * f_noise * i / sample_rate)
    ecg_lms[i] = lms_filter.doFilterAdaptive(ecg_hp[i], noise_ref, learning_rate)

# --- 5. Plot Initial Transient & Two Heartbeat Periods (Task 3 Simple FIR vs Unfiltered) ---
group_delay = (M // 2) / sample_rate      # Linear-phase group delay: (M / 2) / fs = 2.0 s
transient_duration = M / sample_rate      # Full buffer fill time: M / fs = 4.0 s

plt.figure(figsize=(12, 8))
plt.suptitle("Real-Time FIR Filter: Initial Transient and Two-Heartbeat Comparison")

plt.subplot(2, 1, 1)
plt.title(f"Initial Filter Transient (Buffer Fill = {transient_duration:.1f} s, Group Delay = {group_delay:.1f} s)")
plt.plot(t, ecg, alpha=0.5, label="Unfiltered ECG")
plt.plot(t, ecg_filtered, label="Filtered ECG")
plt.axvline(group_delay, color="red", linestyle="--", alpha=0.7, label=f"Group Delay ({group_delay:.1f} s)")
plt.axvline(transient_duration, color="green", linestyle=":", alpha=0.8, label=f"Transient Settled ({transient_duration:.1f} s)")
plt.xlim(0, 6.0)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
plt.grid(True)

beat_start, beat_end = 5.2, 6.8  # ~1.6 s window covering two heartbeat periods (~1.5 Hz heart rate)
in_window = (t >= beat_start) & (t <= beat_end)
ecg_zero_mean = ecg - np.mean(ecg[in_window])

plt.subplot(2, 1, 2)
plt.title("Two Heartbeat Periods: Unfiltered vs Filtered ECG")
plt.plot(t, ecg_zero_mean, alpha=0.5, label="Unfiltered ECG (DC mean subtracted)")
plt.plot(t, ecg_filtered, alpha=0.6, linestyle="--", label="Filtered ECG (with 2.0 s filter delay)")
plt.plot(t - group_delay, ecg_filtered, color="tab:orange", label="Filtered ECG (delay-aligned)")
plt.xlim(beat_start, beat_end)
plt.ylim(-1.2e-4, 2.0e-4)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
```

```
plt.grid(True)

plt.tight_layout()
vector_plot_export("t3_transient_and_beats")

# --- 6. Plot Raw vs Filtered ECG (Time & Frequency Domain Overview) ---
freqs_ecg = np.arange(n // 2) * sample_rate / n
raw_fft = (1 / n) * np.abs(fft(ecg))
filt_fft = (1 / n) * np.abs(fft(ecg_filtered))
hp_fft = (1 / n) * np.abs(fft(ecg_hp))
lms_fft = (1 / n) * np.abs(fft(ecg_lms))

plt.figure(figsize=(12, 8))
plt.suptitle("Real-Time FIR Filtering of ECG Signal")

plt.subplot(3, 1, 1)
plt.title("Time Domain: Raw vs Filtered ECG")
plt.plot(t, ecg, alpha=0.5, label="Raw ECG")
plt.plot(t, ecg_filtered, label="Filtered ECG")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
plt.grid(True)

plt.subplot(3, 1, 2)
plt.title("Zoomed Single Heartbeat (PQRST Complex)")
plt.plot(t, ecg_filtered, color="tab:orange")
plt.xlim(5.5, 6.7)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.grid(True)

plt.subplot(3, 1, 3)
plt.title("Frequency Spectrum: Raw vs Filtered")
plt.plot(freqs_ecg[1:], raw_fft[1:n // 2], alpha=0.5, label="Raw Spectrum")
plt.plot(freqs_ecg[1:], filt_fft[1:n // 2], label="Filtered Spectrum")
plt.xlim(0, sample_rate / 2)
plt.ylim(0, 3e-5)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (V)")
plt.legend(loc="upper right")
plt.grid(True)

plt.tight_layout()
vector_plot_export("t3_filtered")

# --- 7. Plot & Compare Adaptive LMS vs Task 3 Fixed FIR ---
plt.figure(figsize=(12, 8))
plt.suptitle(f"Task 4: Adaptive LMS Filter (M = {M_lms}, $\\mu$ = {learning_rate}) vs Task 3
    Fixed FIR Filter")

plt.subplot(3, 1, 1)
plt.title("Time Domain: Baseline-Corrected ECG vs LMS Filtered vs Fixed FIR")
plt.plot(t, ecg_hp, alpha=0.4, label="HP Only (0-0.5 Hz removed)")
plt.plot(t, ecg_lms, label="Adaptive LMS (50 Hz cancelled)")
plt.plot(t, ecg_filtered, alpha=0.7, linestyle="--", label="Task 3 Fixed FIR")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
plt.grid(True)

plt.subplot(3, 1, 2)
plt.title("Two Heartbeat Periods: Adaptive LMS vs Fixed FIR")
```

```
plt.plot(t, ecg_hp, alpha=0.35, label="HP Only (with 50 Hz hum)")
plt.plot(t, ecg_lms, label="Adaptive LMS")
plt.plot(t, ecg_filtered, linestyle="--", label="Task 3 Fixed FIR")
plt.xlim(beat_start, beat_end)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
plt.grid(True)

plt.subplot(3, 1, 3)
plt.title("Frequency Spectrum Comparison")
plt.plot(freqs_ecg[1:], hp_fft[1:n // 2], alpha=0.4, label="HP Only")
plt.plot(freqs_ecg[1:], lms_fft[1:n // 2], label="Adaptive LMS")
plt.plot(freqs_ecg[1:], filt_fft[1:n // 2], linestyle="--", label="Task 3 Fixed FIR")
plt.xlim(0, sample_rate / 2)
plt.ylim(0, 3e-5)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (V)")
plt.legend(loc="upper right")
plt.grid(True)

plt.tight_layout()
vector_plot_export("t4_compare")

plt.show()
```

C.6 lms_filter.py

```
# 4 - Simulating RT LMS filtering importing fir_filter

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

from fir_design import fir_design
from fir_filter import FIRfilter
from support_funcs import import_ecg_data, vector_plot_export

# --- 1. Load ECG Data ---
t, ecg, sample_rate = import_ecg_data("saad_ecg_2.tsv")
N = len(ecg)

# --- 2. Pre-remove DC / Baseline Wander (0 - 0.5 Hz High-Pass FIR) ---
# Removes DC offset and slow baseline drift so the 50 Hz LMS canceller operates on a zero-mean
    ECG.
h_hp = fir_design(sample_rate, [[0, 0.5]])
hp_filter = FIRfilter(h_hp)

# Fixed FIR filter from Task 3 (for comparison)
h_fixed = fir_design(sample_rate, [[0, 0.5], [45, 125]])
fixed_fir = FIRfilter(h_fixed)

# --- 3. Real-Time Sample-by-Sample LMS Filtering ---
M_lms = 10               # Number of adaptive filter taps (1 period of 50 Hz at 250 Hz is 5
    samples)
learning_rate = 0.01     # LMS step size (mu)
f_noise = 50.0           # 50 Hz mains interference reference frequency

lms = FIRfilter(np.zeros(M_lms))

ecg_hp = np.zeros(N)
ecg_lms = np.zeros(N)
ecg_fixed = np.zeros(N)
```

```
34
35  for n in range(N):
36      # Step 1: High-pass filter raw sample to remove DC / baseline wander
37      ecg_hp[n] = hp_filter.dofilter(ecg[n])
38
39      # Step 2: Generate scalar 50 Hz reference noise and run adaptive LMS step
40      noise = np.sin(2 * np.pi * f_noise * n / sample_rate)
41      ecg_lms[n] = lms.doFilterAdaptive(ecg_hp[n], noise, learning_rate)
42
43      # Task 3 fixed FIR filter output for direct comparison
44      ecg_fixed[n] = fixed_fir.dofilter(ecg[n])
45
46  # --- 4. Plot & Compare Adaptive LMS vs Task 3 Fixed FIR ---
47  freqs = np.arange(N // 2) * sample_rate / N
48  hp_fft = (1 / N) * np.abs(fft(ecg_hp))
49  lms_fft = (1 / N) * np.abs(fft(ecg_lms))
50  fixed_fft = (1 / N) * np.abs(fft(ecg_fixed))
51
52  plt.figure(figsize=(12, 8))
53  plt.suptitle(f"Task 4: Adaptive LMS Filter (M = {M_lms}, $\\mu$ = {learning_rate}) vs Task 3
        Fixed FIR Filter")
54
55  plt.subplot(3, 1, 1)
56  plt.title("Time Domain: Baseline-Corrected ECG vs LMS Filtered vs Fixed FIR")
57  plt.plot(t, ecg_hp, alpha=0.4, label="HP Only (0-0.5 Hz removed)")
58  plt.plot(t, ecg_lms, label="Adaptive LMS (50 Hz cancelled)")
59  plt.plot(t, ecg_fixed, alpha=0.7, linestyle="--", label="Task 3 Fixed FIR")
60  plt.xlabel("Time (s)")
61  plt.ylabel("Amplitude (V)")
62  plt.legend(loc="upper right")
63  plt.grid(True)
64
65  plt.subplot(3, 1, 2)
66  plt.title("Zoomed Single Heartbeat (PQRST Complex Comparison)")
67  plt.plot(t, ecg_hp, alpha=0.35, label="HP Only (with 50 Hz hum)")
68  plt.plot(t, ecg_lms, label="Adaptive LMS")
69  plt.plot(t, ecg_fixed, linestyle="--", label="Task 3 Fixed FIR")
70  plt.xlim(5.5, 6.7)
71  plt.xlabel("Time (s)")
72  plt.ylabel("Amplitude (V)")
73  plt.legend(loc="upper right")
74  plt.grid(True)
75
76  plt.subplot(3, 1, 3)
77  plt.title("Frequency Spectrum Comparison")
78  plt.plot(freqs[1:], hp_fft[1:N // 2], alpha=0.4, label="HP Only")
79  plt.plot(freqs[1:], lms_fft[1:N // 2], label="Adaptive LMS")
80  plt.plot(freqs[1:], fixed_fft[1:N // 2], linestyle="--", label="Task 3 Fixed FIR")
81  plt.xlim(0, sample_rate / 2)
82  plt.ylim(0, 3e-5)
83  plt.xlabel("Frequency (Hz)")
84  plt.ylabel("Magnitude (V)")
85  plt.legend(loc="upper right")
86  plt.grid(True)
87
88  plt.tight_layout()
89  vector_plot_export("t4_compare")
90
91  plt.show()
92
93
94  from support_funcs import import_ecg_data
95
```

```
# --- 1. Load ECG Data ---
t, ecg, sample_rate = import_ecg_data("saad_ecg_2.tsv")
```

C.7 heartbeat.py

```
#5 - simulating real time heartbeat detection using fir_filter

import numpy as np
import matplotlib.pyplot as plt

from fir_design import fir_design
from fir_filter import FIRfilter
from support_funcs import import_ecg_data

#FUNCTIONISE MORE THINGS IN THIS ASSIGNMENT????

#INCLUDE CLEARERR STAGES SEPERATED BY COMMENTS

# Load and format ECG data
t, ecg, sample_rate = import_ecg_data("saad_ecg_2.tsv")  # ECG to analyse
t_template, template_ecg, template_sample_rate = import_ecg_data("will_c_ecg2.tsv")  # other lab partner's ECG


#Apply filter from Task 3
h = fir_design(sample_rate, [[0,0.5], [45, 125]]) #these values were already set previously, seems wasteful to call them again.

#Extract PQRST from TEMPLATE - AI, UNDERSTAND THIS SQUILLIAM
template_fir = FIRfilter(h)
template_clean = np.zeros(len(template_ecg))
for i in range(len(template_ecg)):
        template_clean[i] = template_fir.dofilter(template_ecg[i])


#Cut around the R peak (include QRS). An earlier sample would be affected by cleaning filter transient
R = 2576
template = template_clean[R-12:R+13]
matched_fir = FIRfilter(np.flip(template)) #reverse coefficients in time
ecg_fir = FIRfilter(h) #fresh cleaning filter for ECG

#real time loop (rename this)
matched_fir_out = np.zeros(len(ecg))

THRESHOLD = 1.5e-11 #from matched filter plot
TRANSIENTS = len(template) + len(h) #buffer fill time (cleaning + matched filter) = 1025 samples
MIN = 30
MAX = 200
LAST_BEAT = None #index of previous beat
HEART_RATE = []
BEAT_TIMES = []

for i in range(len(ecg)):
    matched_fir_out[i] = matched_fir.dofilter(ecg_fir.dofilter(ecg[i])) ** 2 #square to make positive (bigger values (peaks) become much bigger))
    if i <= TRANSIENTS:
        continue
    if matched_fir_out[i] > THRESHOLD and matched_fir_out[i-1] <= THRESHOLD:
        peak = i
    if LAST_BEAT != None:
        #Ensure heartrate is within reasonable range (30-200 bpm)
        momentary_rate = (60* sample_rate / peak - LAST_BEAT)
```

```
        in_range =  MIN < momentary_rate < MAX
        hysteresis = abs(momentary_rate - HEART_RATE[-1]) < 0.25 * HEART_RATE[-1] #is that the
    right thing to call it?


print("", len*(heart_rates), np.mean(heart_rates))


# --- ANALYSIS PLOTS ---

#Plot filtered template ecg against sample_number to locate R peak (REMOVE)
plt.plot(template_clean)
plt.xlim(2500, 2800)
plt.xlabel("sample")
plt.show()

#Plot template to check QRS range is covered
plt.plot(template)
plt.xlabel("sample")
plt.ylabel("amplitude")
plt.show()
#plot matched_filter
plt.plot(matched_fir_out)
plt.xlabel("sample")
plt.ylabel("amplitude")
plt.xlim(1000, len(ecg))
plt.ylim(0, 0.5e-10)
plt.show()
```